UI Killshot documentation

Intro

Before I start on the documentation, there are a bunch of odd things happening inside my code that were not there originally, mostly stopping functions that are essential to the flow, I dont know where these scripts continue, but I'm guessing it was some sort of AI code that went through mine and turned things off in bizarre ways rather than just commenting them out without any explanation in the code. I don't want to comb through that and find all the severed wires. Just warning they are there. Described below are all aesthetic UI/art that manipulates the screen, but does nothing to the internal logic, I have no idea why they were switched off. But they worked the last time I saw the game.

Power Node Module



Power node works as a zero sum power resource, move the ticks from the "reactor" to different parts of the ship.

If player finds the power level sufficient for that part of the ship, they turn the charge off.



Player can only decrease the power by venting EVERYTHING and re charging .

If player run out of reactor ticks, player must vent EVERYTHING back into the reactor in order to power other parts of the ship

Player can vent at any time, you can only charge a part of the ship if there are ticks in the reactor.

Functions for outside scripts:

UIController.VentBtn() - vents every power node

UIController.ChargeBtn(int i) - i represents which node is charging: 0-ENG (Engine), 1-WEP (Weapon), 2-CRW (Crew), 3-SHD (shield, not in use), 4-SEN (sensors)

These functions manipulate the “UI_PowerArrow” material which represents where the power is going to and from and “UI_GradeDisplayReactor” material displays the node power level.

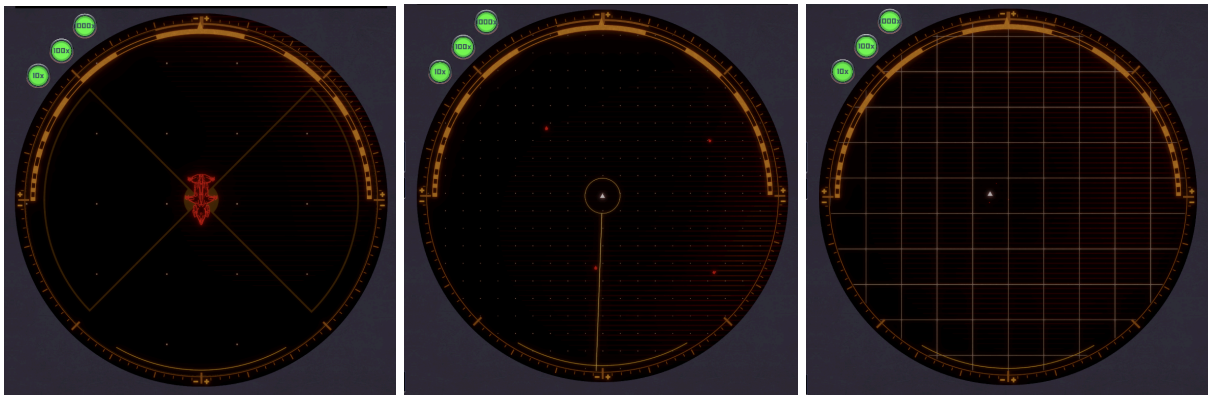
Current Bugs



When the reactor is fully charged, all the “UI_PowerArrow” materials are supposed to stop. Again, this worked perfectly before but somehow no longer functions.

Sometimes when the power node fully fills, it starts to do its emptying routine without starting “UI_PowerArrow” I don’t know how this would work unless it’s calling some other function to do it.

Radare Screen

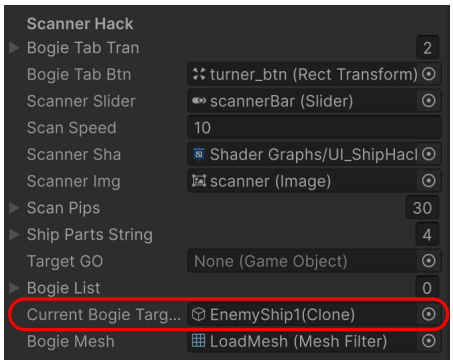


3 zooms heights centered around the ship, [10x] is sonar visibility, [100x] is radar visability, [1000x] is the world space w/ fog of war.

Radar is always present but represents the best at [100x]. bogies(objects in space) are shown

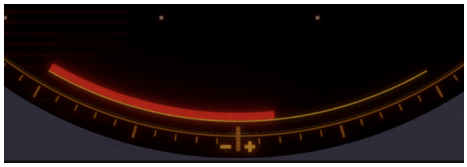
when the radar line points at them in a 360 circle. Regradles on the zoom a red/white/yellow blip is shown on the radius of the screen. I dont think this was ever hooked up, but you can see a test version of the by pressing ”/” in game. Which triggers: UIController.BogeySpot(float), float being the current degree of the radar 0-360. Everytime the radar hits a bogie regardless of the zoom size should trigger this.

Laser Weapons:
Weapons, the side laser weapons need a “currentBogieTarget” and to be within range to show the fire mechanics, the shader won’t show it outside the 2 arcs at the right and left of the ship. If those conditions are meet trigger UIController.LaserFire(). I don’t think this was hooked up properly. But it worked with test cases. Put one of the enemy ships into UIController.currentBogieTarget and drag it into range to test. This is also where you house the information for the scanner to pick up as the player ship can only target one enemy bogie at a time.



Rail Gun:
Fires straight ahead with a small cooldown timer at the beginning, very simple when firing, use UIController.RailFire() for visuals, press “R” to test.

Stability Meter
Located at bottom of radar screen, 0 to 1. Set with UIController.StabilityMeter(float)



Compass
is a +,- degree meter that rotates instead of the ship as the ship will always point UP. should already be hooked up but in case not set world map rotation to UIController.UpdateCompass(float) 0-360.

Turn Raduous
is the checkered arrow around the top half of the circumference forming a crescent moon, it

should turn with the ship and then return to pointing to the top of the screen when the ship stops turning. Can update with UIController.UpdateTurnRadias(float) 0-360.

Comms - Top right module,
Hacking mini game where the random ticks are highlighted, player must hack using the button on the right, or space bar for my test and hit them until all 4 are red. Using UIController.ScanPlaceHack()

Each new scan for a bogie no hacked needs a new random arrangement use UIController.NewScan()

Module Canvas
There are 5 canvases and cameras in the UI to get the video scan line effect, but every UI manipulation happens in the UIController script attached to the PlayerShip. Most of the visual effects are based off shaders to give it those retro graphics. If you have any questions feel free to reach out.